

Design Patterns

Name: DURGASREE AVVARU

Registered Mail: 2300031591cseh@gmail.com

Exercise 1: Singleton Pattern

```
package Design_patterns;

public class Exercise_1 {

    static class Logger {

        private static Logger instance;

        private Logger() {
            System.out.println("Logger Instance Created");
        }

        public static Logger getInstance() {
            if (instance == null) {
                instance = new Logger();
            }
            return instance;
        }

        public void log(String message) {
            System.out.println("LOG: " + message);
        }
    }

    public static void main(String[] args) {

        Logger logger1 = Logger.getInstance();
        Logger logger2 = Logger.getInstance();

        logger1.log("First Message");
        logger2.log("Second Message");
    }
}
```

```

        if (logger1 == logger2) {
            System.out.println("Only one Logger instance exists.");
        } else {
            System.out.println("Multiple Logger instances exist.");
        }
    }
}

```

Output Screenshot

The screenshot shows the Eclipse IDE with a Java project. The editor displays the following code:

```

11     }
12
13     public static Logger getInstance() {
14         if (instance == null) {
15             instance = new Logger();
16         }
17         return instance;
18     }
19
20     public void log(String message) {
21         System.out.println("LOG: " + message);
22     }
23 }
24
25 public static void main(String[] args) {
26
27     Logger logger1 = Logger.getInstance();
28     Logger logger2 = Logger.getInstance();
29
30     logger1.log("First Message");
31     logger2.log("Second Message");
32
33     if (logger1 == logger2) {
34         System.out.println("Only one Logger instance exists.");
35     }
36 }

```

The Console window at the bottom shows the following output:

```

<terminated> Exercise_1 [Java Application] C:\Users\lokak\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v202507
Logger Instance Created
LOG: First Message
LOG: Second Message
Only one Logger instance exists.

```

Exercise 2: Factory Method Pattern

```
package Design_patterns;
```

```
public class Exercise_2 {
```

```
    interface Document { void open(); }
```

```

    static class WordDocument implements Document {
        public void open() { System.out.println("Word Document Opened"); }
    }
}

```

```

static class PdfDocument implements Document {
    public void open() { System.out.println("PDF Document Opened"); }
}

static class ExcelDocument implements Document {
    public void open() { System.out.println("Excel Document Opened"); }
}

static abstract class DocumentFactory {
    public abstract Document createDocument();
}

static class WordFactory extends DocumentFactory {
    public Document createDocument() { return new WordDocument(); }
}

static class PdfFactory extends DocumentFactory {
    public Document createDocument() { return new PdfDocument(); }
}

static class ExcelFactory extends DocumentFactory {
    public Document createDocument() { return new ExcelDocument(); }
}

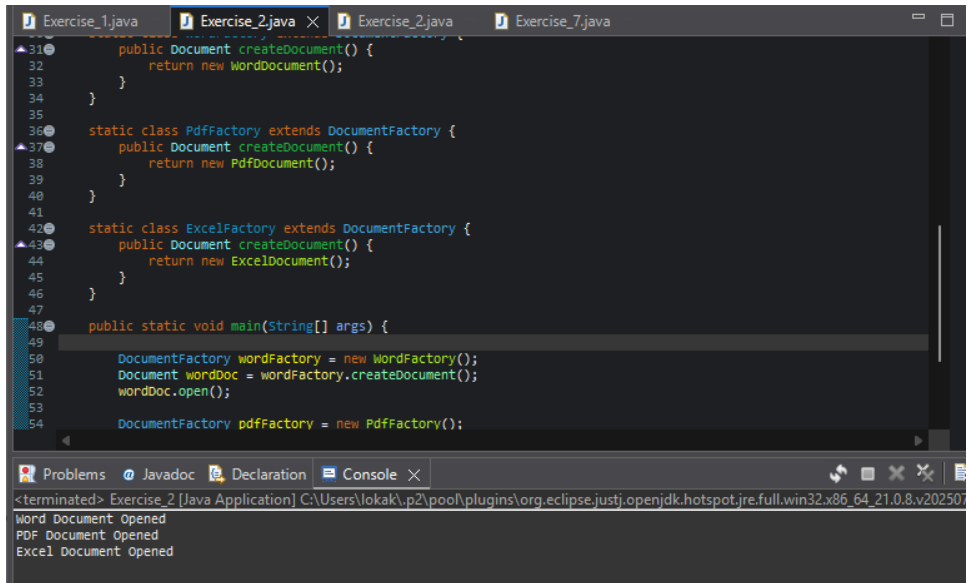
public static void main(String[] args) {
    DocumentFactory wordFactory = new WordFactory();
    Document wordDoc = wordFactory.createDocument();
    wordDoc.open();

    DocumentFactory pdfFactory = new PdfFactory();
    Document pdfDoc = pdfFactory.createDocument();
    pdfDoc.open();

    DocumentFactory excelFactory = new ExcelFactory();
    Document excelDoc = excelFactory.createDocument();
    excelDoc.open();
}
}

```

Output Screenshot:



The screenshot displays the Eclipse IDE interface. The top editor shows the code for `Exercise_2.java`, which implements a Factory Method pattern. It defines a base class `DocumentFactory` with a `createDocument()` method. Two subclasses, `PdfFactory` and `ExcelFactory`, override this method to return `PdfDocument` and `ExcelDocument` respectively. The `main` method creates instances of `WordFactory` and `PdfFactory`, and demonstrates the `createDocument()` method.

```
31 public Document createDocument() {
32     return new WordDocument();
33 }
34 }
35
36 static class PdfFactory extends DocumentFactory {
37     public Document createDocument() {
38         return new PdfDocument();
39     }
40 }
41
42 static class ExcelFactory extends DocumentFactory {
43     public Document createDocument() {
44         return new ExcelDocument();
45     }
46 }
47
48 public static void main(String[] args) {
49
50     DocumentFactory wordFactory = new WordFactory();
51     Document wordDoc = wordFactory.createDocument();
52     wordDoc.open();
53
54     DocumentFactory pdfFactory = new PdfFactory();
```

The bottom console window shows the output of the program:

```
<terminated> Exercise_2 [Java Application] C:\Users\lokak\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v202507
Word Document Opened
PDF Document Opened
Excel Document Opened
```